

Unit 07: Standards

CONTENTS

Objectives

Introduction

7.1 Common Errors

7.2 Programming Practices

7.3 Coding Standards

7.4 Code Reusability

7.5 Documentation

Summary

Keywords

Self Assessment

Answer for Self Assessment

Review Questions

Further Reading

Objectives

After studying this unit, you will be able to:

- Good coding practices
- Coding standards and code reusability
- Documentation standards

Introduction

The process of converting a system's design into a computer language format is known as coding. The coding step of software development is responsible for converting design specifications into source code. The task of coding is to convert design specifications into source code. A coding standard ensures that code generated by various engineers has a consistent appearance. It facilitates the understanding of the code. Encourages the use of excellent programming practices.

Debugging, testing, and change should be simple with proper programming. This is accomplished by making the source code as simple and transparent as feasible. "Simple is great," says an old adage. The trademarks of good programmes are simplicity, clarity, and elegance. Inadequate design is characterised by obscurity, cunning, and complexity. Structured coding practices, decent coding style, adequate supporting documents, good internal comments, and other factors improve source code clarity. Production of high quality software requires that the programming team should have a thorough understanding of duties and responsibilities and should be provided with a well-defined set of requirements, an architectural design specification, and a detailed design description.

Certain principles must be followed when creating and implementing software or apps to ensure that your programme satisfies the needed attributes to perform as intended. These standards also ensure that the final output meets your clients' expectations. The ability to build computer software, apps, and websites is made possible through coding. The coding step of the software development life cycle is the longest. Coding normally begins when the design documentation is complete. This phase will almost totally be devoted to the developer's efforts. The development of code that will implement the design is the focus of the coding phase of the software life-cycle. This code is written in a

programming language, which is a formal language. Programming languages have progressed over time from sequences of ones and zeros that a computer can directly parse, to symbolic machine code, assembly languages, and finally higher-level languages that humans can understand.

7.1 Common Errors

A mismatch between the programme and its specification is a common definition of a software error. To put it another way, a software error occurs when a programme fails to perform as expected by its end user. Depending on the type of programme and the specific bug involved, errors might result in a wide range of issues.

Others have the potential to cause errors in the performance of the program that result in the program behaving in unexpected ways. Sometimes a software error can even cause a program to shut down completely.

Following are the most common software errors. This helps you to identify errors systematically and increases the efficiency and productivity of software testing.

Syntax error	Logical error
Runtime error	Enumerated error
Memory leaks	Null dereferencing
Synchronization error	String handling errors
Lack of unique addresses	freeing an already freed resource

Example: We have discussed below different types of errors with examples:

Syntax error: A syntax error is an error in the source code of a program. Since computer programs must follow strict syntax to compile correctly, any aspects of the code that do not conform to the syntax of the programming language will produce a syntax error.

Syntax errors are small grammatical mistakes, sometimes limited to a single character. For example, a missing semicolon at the end of a line or an extra bracket at the end of a function may produce a syntax error.

Logical error: A logic error (or logical error) is a 'bug' or mistake in a program's source code that results in incorrect or unexpected behaviour. It is a type of runtime error that may simply produce the wrong output or may cause a program to crash while running. Many different types of programming mistakes can cause logic errors. For example, assigning a value to the wrong variable may cause a series of unexpected program errors. Multiplying two numbers instead of adding them together may also produce unwanted results.

Runtime error: If there are no syntax errors, Java may detect an error while your program is running. You will get an error message telling you the kind of error, and a stack trace that tells not only where the error occurred, but also what other method or methods you were in.

User Interface Errors: Missing/Wrong Functions, Doesn't do what the user expects, Missing information, Misleading, Confusing information, Wrong content in Help text, Inappropriate error messages. Performance issues - Poor responsiveness, Can't redirect output, inappropriate use of key board.

Error Handling: Inadequate – protection against corrupted data, tests of user input, version control; Ignores – overflow, data comparison, Error recovery – aborting errors, recovery from hardware problems.

Boundary Related Errors: Boundaries in loop, space, time, memory, mishandling of cases outside boundary.

Calculation Errors: Bad Logic, Bad Arithmetic, Outdated constants, Calculation errors, incorrect conversion from one data representation to another, Wrong formula, incorrect approximation.

Initial and Later States: Failure to – set data item to zero, to initialize a loop-control variable, or re-initialize a pointer, to clear a string or flag, Incorrect initialization.

Control Flow Errors: Wrong returning state assumed, Exception handling based exits, Stack underflow/overflow, Failure to block or unblock interrupts, Comparison sometimes yields wrong result, Missing/wrong default, Data Type errors.

Errors in Handling or Interpreting Data: Unterminated null strings, overwriting a file after an error exit or user abort.

Race Conditions: Assumption that one event or task finished before another begins,

Resource races, Tasks starts before its prerequisites are met, Messages cross or don't arrive in the order sent.

Load Conditions: Required resources are not available, No available large memory area, Low priority tasks not put off, doesn't erase old files from mass storage, and doesn't return unused memory.

Hardware: Wrong Device, Device unavailable, underutilizing device intelligence, Misunderstood status or return code, Wrong operation or instruction codes.

Source, Version and ID Control: No Title or version ID, Failure to update multiple copies of data or program files.

Testing Errors: Failure to notice/report a problem, Failure to use the most promising test case, Corrupted data files, Misinterpreted specifications or documentation, Failure to make it clear how to reproduce the problem, Failure to check for unresolved problems just before release, Failure to verify fixes, Failure to provide summary report.

7.2 Programming Practices

General coding guidelines provide the programmer with a set of best practices which can be used to make programs easier to read and maintain. Unlike the coding standards, the use of these guidelines is not mandatory. However, the programmer is encouraged to review them and attempt to incorporate them into his/her programming style where appropriate. Most of the examples use the C language syntax but the guidelines can be applied to all languages.

Line Length

It is considered good practice to keep the lengths of source code lines at or below 80 characters.

Lines longer than this may not be displayed properly on some terminals and tools. Some printers will truncate lines longer than 80 columns. FORTRAN is an exception to this standard.

Its line lengths cannot exceed 72 columns.

Spacing

The proper use of spaces within a line of code can enhance readability. Good rules of thumb are as follows:

A keyword followed by a parenthesis should be separated by a space.

A blank space should appear after each comma in an argument list.

All binary operators except "." should be separated from their operands by spaces.

Blank spaces should never separate unary operators such as unary minus, increment ("++"), and decrement ("--") from their operands.

Casts should be made followed by a blank space.

Example: Bad:

```
cost=price+ (price*sales tax);
fprintf (stdout," The total cost is %5.2f\n", cost);
```

Better:

```
cost = price + (price * sales tax);
fprintf (stdout, "The total cost is %5.2f\n", cost);
```

Wrapping Lines

When an expression will not fit on a single line, break it according to these following principles:

Break after a Comma

Example: `fprintf (stdout, "\nThere are %d reasons to use standards\n", num_reasons);`

Break after an Operator

Example: `long int total_apples = num_my_apples + num_his_apples +
num_her_apples;`

Prefer Higher-level Breaks to Lower-level Breaks

Example: Bad:

`longName1 = longName2 * (longName3 + LongName4 - longName5) + 4 * longName6;`

Better:

`longName1 = longName2 * (longName3 + LongName4 - LongName5) + 4 * longName6;`

Align the new line with the beginning of the expression at the same level on the previous line.

Example: `total_windows = number_attic_windows + number_second_floor_windows
+ number_first_floor_windows;`

Variable Declarations

Variable declarations that span multiple lines should always be preceded by a type.

Example: Acceptable:

`int price, score;`

Acceptable:

`int price;`

`int score;`

Not Acceptable:

`int price,`

`score;`

Program Statements

Program statements should be limited to one per line. Also, nested statements should be avoided when possible.

Example: Bad:

`number_of_names = names.length; b = new JButton [ number_of_names];`

Better:

`number_of_names = names.length;`

`b = new JButton [ number_of_names];`

Use of Parentheses

It is better to use parentheses liberally. Even in cases where operator precedence unambiguously dictates the order of evaluation of an expression, often it is beneficial from a readability point of view to include parentheses anyway.

In-line Comments

In-line comments promote program readability. They allow a person not familiar with the code to more quickly understand it. It also helps the programmer who wrote the code to remember details

forgotten over time. This reduces the amount of time required to perform software maintenance tasks.

As the name suggests, in-line comments appear in the body of the source code itself. They explain the logic or parts of the algorithm which are not readily apparent from the code itself. In-line comments can also be used to describe the task being performed by a block of code.

In-line comments should be used to make the code clearer to a programmer trying to read and understand it. Writing a well-structured program lends much to its readability even without inline comments. The bottom line is to use in-line comments where they are needed to explain complicated program behavior or requirements. Use in-line comments to generalize what a block of code, conditional structure, or control structure is doing. Do not use overuse in-line comments to explain program details which are readily obvious to an intermediately skilled programmer.

Coding for Efficiency vs. Coding for Readability

There are many aspects to programming. These include writing software that runs efficiently and writing software that is easy to maintain. These two goals often collide with each other.

Creating code that runs as efficiently as possible often means writing code that uses tricky logic and complex algorithms, code that can be hard to follow and maintain even with ample in-line comments.

If a more complicated algorithm offers only small gains in the speed of a program, the programmer should consider using a simpler algorithm. Although slower, the simpler algorithm will be easier for other programmers to understand.

Meaningful Error Messages

Error handling is an important aspect of computer programming. This not only includes adding the necessary logic to test for and handle errors but also involves making error messages meaningful.

Error messages should be meaningful. When possible, they should indicate what the problem is, where the problem occurred, and when the problem occurred. A useful Java exception handling feature is the option to show a stack trace, which shows the sequence of method calls which led up to the exception.

Code which attempts to acquire system resources such as dynamic memory or files should always be tested for failure.

Error messages should be stored in way that makes them easy to review. For non-interactive applications, such as a program which runs as part of a cron job, error messages should be logged into a log file. Interactive applications can either send error messages to a log file, standard output, or standard error. Interactive applications can also use popup windows to display error messages.

Reasonably Sized Functions and Methods

Software modules and methods should not contain an excessively large number of lines of code. They should be written to perform one specific task. If they become too long, then chances are the task being performed can be broken down into sub-tasks which can be handled by new routines or methods.

A reasonable number of lines of code for routine or a method is 200. This does not include documentation, either in the function/method header or in-line comments.

Number of Routines per File

It is much easier to sort through code you did not write and you have never seen before if there are a minimal number of routines per file. This is only applicable to procedural languages such as C and FORTRAN. It does not apply to C++ and Java where there tends to be one public class definition per file.

7.3 Coding Standards

Coding standards help to ensure safety, security, and reliability. Coding standards are collections of coding rules, guidelines, and best practices. Compliance with industry standards (e.g., ISO). Consistent code quality — no matter who writes the code. Software security from the start. Reduced development costs and accelerated time to market.

Coding standards is a series of procedures that can be defined for a particular programming language specifying a programming style, the methods and different procedures. A coding standard makes sure that all the developers working on the project are following certain specified guidelines.

Objective of Coding Standards

A coding standard gives a uniform appearance to the codes written by different engineers. It improves readability, and maintainability of the code and it reduces complexity also. It helps in code reuse and helps to detect error easily. It promotes sound programming practices and increases efficiency of the programmers.

- Limited use of globals
- Standard headers for different modules
- Naming conventions for local variables, global variables, constants and functions
- Indentation
- Error return values and exception handling conventions
- Avoid using a coding style that is too difficult to understand
- Avoid using an identifier for multiple purposes
- Code should be well documented
- Length of functions should not be very large
- Try not to use GOTO statement

Rules for limiting the use of global: These rules list what types of data can be declared global and what cannot.

Standard headers for different modules: The header of different modules should follow some standard format and information

Name of the module

Date of module creation

Author of the module

Modification history

Different functions supported in the module along with their input output parameters

Global variables accessed or modified by the module

Naming conventions for local and global variables, constants and functions: Meaningful and understandable variables name helps anyone to understand the reason of using it

Need to follow case sensitive rule as per language

It is better to avoid the use of digits in variable names.

Indentation: There must be a space after giving a comma between two function arguments.

Each nested block should be properly indented and spaced.

Proper Indentation should be there at the beginning and at the end of each block in the program.

All braces should start from a new line and the code following the end of braces also start from a new line.

Error return values and exception handling conventions: The way error conditions are reported by different functions in a program are handled should be standard within an organization.

For example, different functions while encountering an error condition should either return a 0 or 1 consistently.

Avoid using a coding style that is too difficult to understand: Code should be easily understandable. The complex code makes maintenance and debugging difficult and expensive.

Code should be well documented: The code should be properly commented for understanding easily. Comments regarding the statements increase the understandability of the code.

Length of functions should not be very large: Lengthy functions are very difficult to understand. That's why functions should be small enough to carry out small work and lengthy functions should be broken into small ones for completing small tasks.

Try not to use GOTO statement: GOTO statement makes the program unstructured, thus it reduces the understandability of the program and also debugging becomes difficult.

Classes, Subroutines, Functions, and Methods: Keep subroutines, functions, and methods reasonably sized. This depends upon the language being used. A good rule of thumb for module length is to constrain each module to one function or action (i.e. each module should only do one "thing"). If a module grows too large, it is usually because the programmer is trying to accomplish too many actions at one time.

The names of the classes, subroutines, functions, and methods shall have verbs in them.

That is the names shall specify an action, e.g. "get name", "compute temperature".

Source File: The name of the source file or script shall represent its function. All of the routines in a file shall have a common purpose.

Use of Braces: In some languages, braces are used to delimit the bodies of conditional statements, control constructs, and blocks of scope. Programmers shall use either of the following bracing styles:

```
for (int j = 0; j < max_iterations; ++j)
{
    /* Some work is done here. */
}
```

or the Kernighan and Ritchie style:

```
for (int j = 0; j < max_iterations; ++j) {
    /* Some work is done here. */
}
```

It is felt that the former brace style is more readable and leads to neater-looking code than the latter style, but either use is acceptable.

Compiler Warnings: Compilers often issue two types of messages: warnings and errors. Compiler warnings normally do not stop the compilation process. However, compiler errors do stop the compilation process, forcing the developer to fix the problem and recompile. Compiler and linker warnings shall be treated as errors and fixed. Even though the program will continue to compile in the presence of warnings, they often indicate problems which may affect the behavior, reliability and portability of the code. Some compilers have options to suppress or enhance compile-time warning messages. Developers shall study the documentation and/or man pages associated with a compiler and choose the options which fully enable the compiler's code-checking features.

Advantages of Coding Guidelines

- Coding guidelines increase the efficiency of the software and reduces the development time.
- Coding guidelines help in detecting errors in the early phases, so it helps to reduce the extra cost incurred by the software project.
- If coding guidelines are maintained properly, then the software code increases readability and understandability thus it reduces the complexity of the code.
- It reduces the hidden cost for developing the software.

Example: Coding standards

IEC 61508: "Functional safety of electrical/ electronic /programmable electronic safety-related systems"

ISO 26262: "Road vehicles – functional safety"

EN 50128: "Railway applications – Communication, signaling, and processing systems – Software for railway control and protection systems"

IEC 62061: "Safety of machinery: Functional safety of electrical, electronic and programmable electronic control systems"

MISRA

MISRA provides coding guidelines for C and C++.

The MISRA C coding standard was originally written for the automotive embedded software industry.

MISRA standards for C and C++ are widely used by embedded industries – including aerospace and defense, telecommunications, medical devices, and rail.

JSFC++

Joint Strike Fighter Air Vehicle C++ (JSF AV C++) is a coding standard developed by Lockheed Martin.

It helps programmers develop error-free code for safety-critical systems.

AUTOSAR

The AUTomotive Open System ARchitecture (AUTOSAR) was founded in order to standardize the functionality of automotive embedded software

AUTOSAR is a worldwide partnership that is comprised of over 180 automotive manufacturers, automotive suppliers, tool vendors, semiconductor vendors.

CERT

An estimated 82% of software vulnerabilities are caused by coding errors.

CERT is a secure coding standard that supports commonly used programming languages such as C, C++, and Java.

The standards are developed through a broad-based community effort.

The rules and recommendations target insecure coding practices and undefined behaviors that lead to security risks.

7.4 Code Reusability

Code reusability defines the methodology you can use to use similar code, without having to re-write it everywhere. There are many techniques to make code reusable in your applications. This technique follows the general programming philosophy of Don't Repeat Yourself (DRY).

Code can be reused when it can be

Easily extended and adapted for the new application. Ported to different hardware if needed. Shown to be free from defects or problems that affect the reliability, safety, or security of the new application.

There are many scenarios and methods

- Inheritance
- Functions
- Libraries
- Forking

Inheritance:

Inheritance has been in the programming for quite a long time and is used in Object-Oriented Programming paradigms. Inheritance lets you use the base class's functions and members in other classes.

Functions:

A return type that denotes what type of data would be returned or a void. An identifier that is used to make a call to this function. Function body that is the code block containing the code to be executed when this function is called. Zero or more parameters to provide values to the code block variables. Function calls are another way of reusing the code from various parts of your application. You create small code blocks of an algorithm or logic-based source code and provide it a name, identifier.

Libraries:

Libraries are a set of compiled programs, most often used by larger and enterprise level software, to support the execution since they can be executed upon call.

Forking:

Forking is a process of using someone else's source code and making changes to it to fit your own needs, to distribute, use, and share and so on. User can also fork someone else's source code and write some more details to it. Make some bug fixes, add some patches and share it.

This method has been widely used by various companies, to fork open source projects, make a few changes to the "Under the hood" and then distribute it with their own label and tag.

7.5 Documentation

All these documents are a vital part of good software development practice. Documents such as users' manual, software requirements specification (SRS) documents, design documents, test documents, installation manual, etc are also developed as part of any software engineering process.

Significance of Documentation

Good documents enhance understandability and maintainability of a software product. They reduce the effort and time required for maintenance. Documents help the users in effectively using the system. Good documents help in effectively handling the manpower turnover problem. Production of good documents helps the manager in effectively tracking the progress of the project.

Types of documents

- User manual
- Operational manual
- Requirements Document
- Design document
- Technical document
- Testing document
- List of Known Bugs

User manual – It describes instructions and procedures for end users to use the different features of the software.

Operational manual – It lists and describes all the operations being carried out and their inter-dependencies.

Design Document – It gives an overview of the software and describes design elements in detail. It documents details like data flow diagrams, entity relationship diagrams.

Requirements Document – It has a list of all the requirements of the system as well as an analysis of viability of the requirements. It can have user cases, real life scenarios, etc.

Technical Documentation – It is a documentation of actual programming components like algorithms, flowcharts, program codes, functional modules, etc.

Testing Document – It records test plan, test cases, validation plan, verification plan, test results, etc. Testing is one phase of software development that needs intensive documentation.

Internal documentation is also provided through the useful variable names, module and function headers, code indentation, code structuring, use of enumerated types and constant identifiers, use of user-defined data types, etc.

External documentation - It is provided through various types of supporting documents such as users' manual, software requirements specification document, design document, test documents, etc. A systematic software development style ensures that all these documents are produced in an orderly fashion.

- **Guidelines for creating the documents**
- Documentation should be from the point of view of the reader
- Document should be unambiguous
- There should be no repetition
- Industry standards should be used
- Documents should always be updated
- Any outdated document should be phased out after due recording of the phase out

Advantages of Documentation

- Keeps track of all parts of a software or program
- Maintenance is easier
- Programmers other than the developer can understand all aspects of software
- Improves overall quality of the software
- Assists in user training
- Ensures knowledge de-centralization, cutting costs and effort if people leave the system abruptly

Summary

- A software error is present in a program when the program does not do what its end user expects.
- Errors can cause a wide variety of different problems depending on the kind of program and the particular kind of bug involved.
- Structured Programming is a method of planning programs that avoids the branching category of control structures.
- The single most important idea of structured programming is that the code you write should represent a clear, simple and straightforward solution to the problem at hand.
- General coding guidelines provide the programmer with a set of best practices which can be used to make programs easier to read and maintain.
- In-line comments promote program readability. They allow a person not familiar with the code to more quickly understand it.
- If a more complicated algorithm offers only small gains in the speed of a program, the programmer should consider using a simpler algorithm.
- Compiler warnings normally do not stop the compilation process. However, compiler errors do stop the compilation process, forcing the developer to fix the problem and recompile.

Keywords

Compiler Errors: Compiler errors do stop the compilation process, forcing the developer to fix the problem and recompile.

Compiler Warnings: Compiler warnings is a message which normally do not stop the compilation process.

Error Handling: Error handling includes adding the necessary logic to test for and handle errors but also involves making error messages meaningful.

In-line Comments: In-line comments allow a person not familiar with the code to more quickly understand it.

Sequence: Sequence is a series of program statements are executed one after the other, in the order in which they appear in the source code.

Series: A series of statements is executed repeatedly until some termination condition is met.

Software Error: A software error is a mismatch between the program and its specification.

Structured Programming: Structured Programming is a method of planning programs that avoids the branching category of control structures.

Self Assessment

1. What are the common types of error in coding?
 - A. Memory leaks
 - B. Null dereferencing
 - C. Null dereferencing
 - D. All of above

2. Which is not a syntax error?
 - A. `printf ("this is code");`
 - B. `printf ("this is code");`
 - C. `printf ('this is code');`
 - D. none of above

3. Which is not a coding error?
 - A. Logical error
 - B. Lack of unique addresses
 - C. Transformation
 - D. None of the above

4. A good coding practice is?
 - A. Portability
 - B. Generality
 - C. Enhance Code Readability
 - D. All of above

5. Coding is____
 - A. To translate the design of system into a computer language format:
 - B. To reduce the cost of later phases
 - C. Making the program more readable
 - D. All of above

6. Coding standards are__
 - A. Help to ensure safety, security, and reliability.
 - B. Collections of coding rules, guidelines, and best practices
 - C. Compliance with industry standards
 - D. All of above

7. Which is not a part of coding standard?
 - A. ISO 2010
 - B. IEC 61508
 - C. EN 50128
 - D. IEC 62061

8. Which coding standard is for Railway applications?
 - A. ISO 26262
 - B. IEC 62061
 - C. EN 50128
 - D. None of the above

9. Lockheed Martin developed____
 - A. MISRA
 - B. JSFC++
 - C. AUTOSAR
 - D. All of above

10. MISRA coding standard is used for__
 - A. Fighter Air Vehicle
 - B. Automotive embedded software industry
 - C. Toy industry
 - D. None of above

11. What are the different type of documents?
 - A. Operational manual
 - B. List of Known Bugs
 - C. User manual
 - D. All of above

12. Code can be reuse in case_
 - A. Easily extended and adapted for the new application.
 - B. Ported to different hardware if needed.
 - C. Shown to be free from defects
 - D. All of above

13. Which is not a Code reusability methods?
 - A. Inheritance
 - B. Functions
 - C. Classes
 - D. Forking

14. What are different Code reusability methods?
 - A. Functions
 - B. Libraries
 - C. Forking
 - D. All of above

15. Function call is part of ____
 - A. Testing
 - B. Classes
 - C. Code reusability
 - D. None of above

Answer for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. D | 2. B | 3. C | 4. D | 5. D |
| 6. D | 7. A | 8. C | 9. B | 10. B |
| 11. D | 12. D | 13. C | 14. D | 15. C |

Review Questions

1. Discuss different types of common errors with example.
2. Explain the technique used for organizing and coding computer programs.
3. Discuss the basic principles of structured programming.
4. Describe the building blocks of structured programming.
5. What are the different types of control blocks? Discuss.
6. Discuss the best programming practices which can be used to make programs easier to read and maintain.
7. Illustrate how the use of spaces within a line of code can enhance readability.
8. Discuss the general coding standards used by the developer when writing code.



Further Reading

- Books Rajib Mall, Fundamentals of Software Engineering, 2nd Edition, PHI.
- Richard Fairpy, Software Engineering Concepts, Tata McGraw Hill, 1997.
- R.S. Pressman, Software Engineering – A Practitioner's Approach, 5th Edition, Tata

- McGraw Hill Higher education.
- Sommerville, Software Engineering, 6th Edition, Pearson Education



- <http://software-engineering142.blogspot.in/2009/04/coding-phase.html>
- http://www.computer.org/portal/web/certification/resources/code_of_ethics
- <http://www.indeed.com/q-Software-Process-Engineer-Coding-StandardPractice-jobs.html>
- http://www.win.tue.nl/~wstomv/edu/jpid/Coding_Standard_Motivation-4up.pdf